



Locking



Content

- **Issues in concurrency control**
- Locking schema
 - Introduction
 - Two phase locking
 - Lock modes

Issues in concurrency control

- (1) Lost updated
- (2) Unrepeatable read
- (3) “Phantom”
- (4) Dirty read

Lost updated

■ Examine	T_1	T_2
	Read(A)	Read(A)
	$A := A + 10$	$A := A + 20$
	Write(A)	Write(A)

- T_1 and T_2 are performed concurrently

A=50	T_1	T_2
t_1	Read(A)	
t_2		Read(A)
t_3	$A := A + 10$	
t_4		$A := A + 20$
t_5	Write(A)	
t_6		Write(A)

A=60

A=70

- The value of A modified at t_4 by T_1 is lost
- It is overwritten at t_6

What happens if T_2 writes an incorrect value ???

Unrepeatable read

■ Examine	T ₁	T ₂
	Read(A)	Read(A)
	A:=A+10	Print(A)
	Write(A)	Read(A)
		Print(A)

- T1 and T2 are performed concurrently

A=50	T ₁	T ₂	
t ₁	Read(A)		
t ₂		Read(A)	A=50
t ₃	A:=A+10		
t ₄		Print(A)	A=50
t ₅	Write(A)		
t ₆		Read(A)	A=60
t ₇		Print(A)	A=60

- T₂ read A two times
- Results of these times are different

Phantom

- Examine T_1 and T_2 as follows
 - A and B are balances of banking accounts
 - T_1 transfers money from A to B
 - T_2 checks the sum of A and B

A=70, B=50	T_1	T_2	
t_1	Read(A)		A=70
t_2	A:=A-50		
t_3	Write(A)		A=20
t_4		Read(A)	A=20
t_5		Read(B)	B=50
t_6		Print(A+B)	A+B=70
t_7	Read(B)		
t_8	B:=B+50		
t_9	Write(B)		

Lost 50 ???

Dirty read

`Transfer(account1, account2, money)`

1. Add money to account2

2. Test if account1 has enough money

2a. Not enough, remove money from account2

2b. Enough, subtract money from account1

	T_1 transfers 150 from A_1 to A_2	T_2 transfer 250 from A_2 to A_3	$A_1=100$	$A_2=200$	$A_3=300$	600
t_1		$A_3 := A_3 + 250$			550	
t_2	$A_2 := A_2 + 150$			350		
t_3		Test $A_2 \geq 250$?				Dirty read
t_4	Test $A_1 \geq 150$?					
t_5		$A_2 := A_2 - 250$		100		
t_6	$A_2 := A_2 - 150$			-50		600

Dirty read

- Examine T_1 and T_2 as follows

	T_1	T_2
t_1	Read(A)	
t_2	$A := A + 10$	
t_3	Write(A)	
t_4		Read(A)
t_5		Print(A)
t_6	Abort	

- T_2 reads the value of A written by T_1
- But, T_1 aborts

Dirty read

Discussion

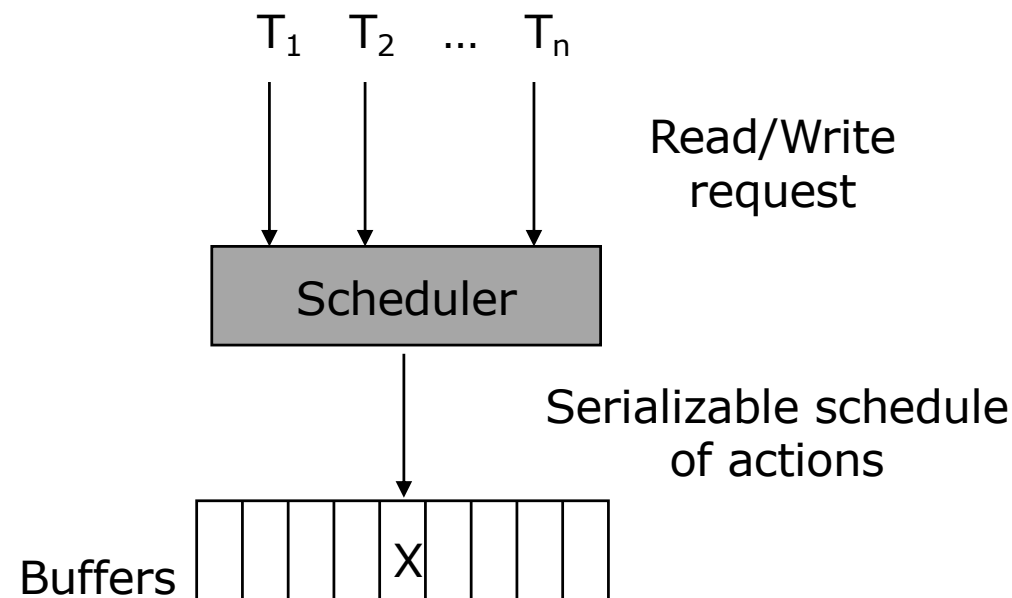
- A scheduler prevents
 - Unserializable schedules
 - Issues in concurrency control
- In reality
 - What will commercial DBMSs do?

Content

- Issues in concurrency control
- **Locking schema**
 - Introduction
 - Two phase locking
 - Lock modes

Remind

- Responsibility of scheduler
 - Take requests from transactions
 - Either allow requests to operate on the database
 - Or defer requests for a while
 - Until it is safe to allow requests to execute



Discussion

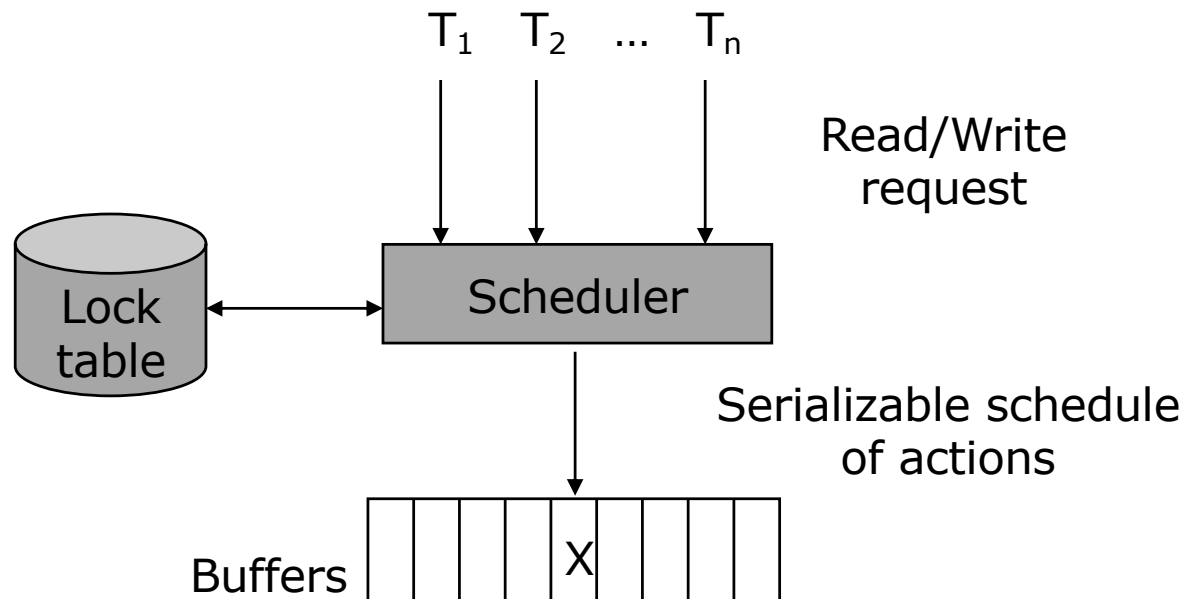
- When are these requests operated?
- When are these requests deferred?
- How to prevent a cyclic $P(S)$ at the time of execution?

Idea

- Instead enforce conflict-serializability, schedulers use a simple test
 - Guarantee serializability
 - May forbid some actions that could lead to inconsistency
- ***Locking*** scheduler
- Note
 - More stringent condition than serializability

Locking scheduler

- Add two more actions
 - Lock action
 - Unlock action



Locking scheduler

- Transactions must request a ***lock*** on a database element X before read/write
 - Lock(X) or l(X)
- This request is managed by ***lock manager***
 - If it is accepted, transactions will be allowed to read/write on database elements
- After read/write, transactions must release (***unlock***) the database elements
 - Unlock(X) or u(X)

Locking scheduler

- The use of lock must be proper in two senses

- (1) *Consistency of transactions*

$T_i :$... **$l(X)$** ... $r(X)/w(X)$... **$u(X)$** ...

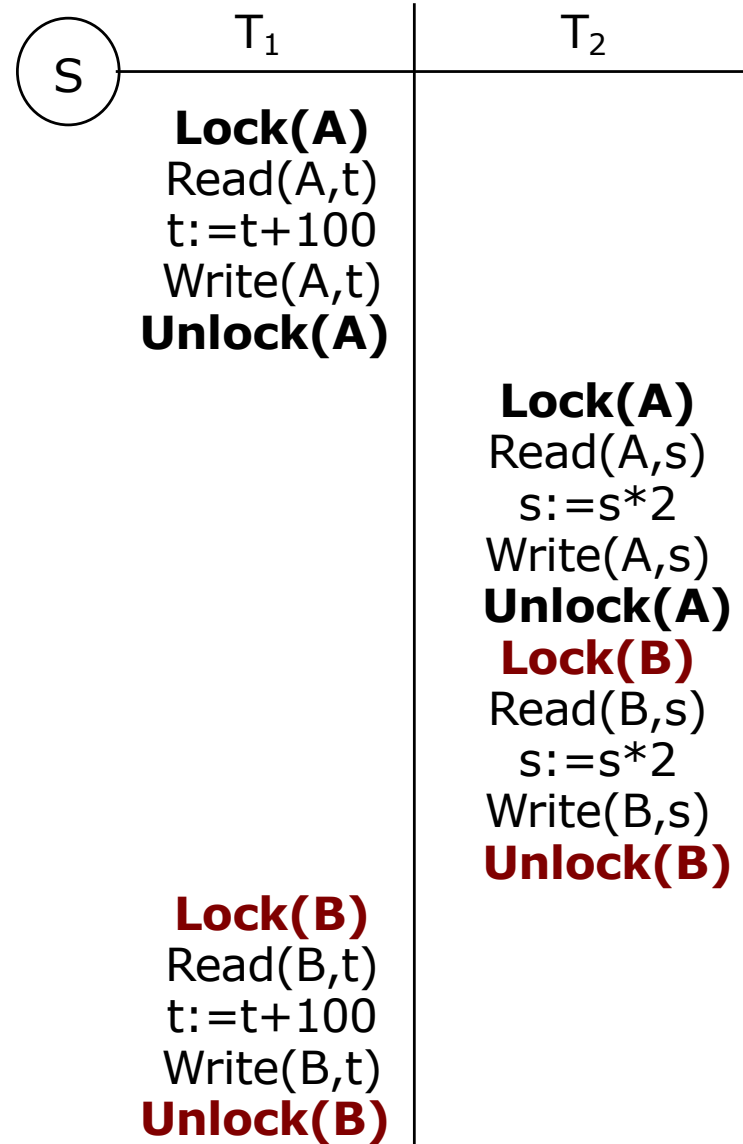
- (2) *Legality of schedules*

$S :$... $l_i(X)$ $u_i(X)$...



no **$l_j(X)$**

Example



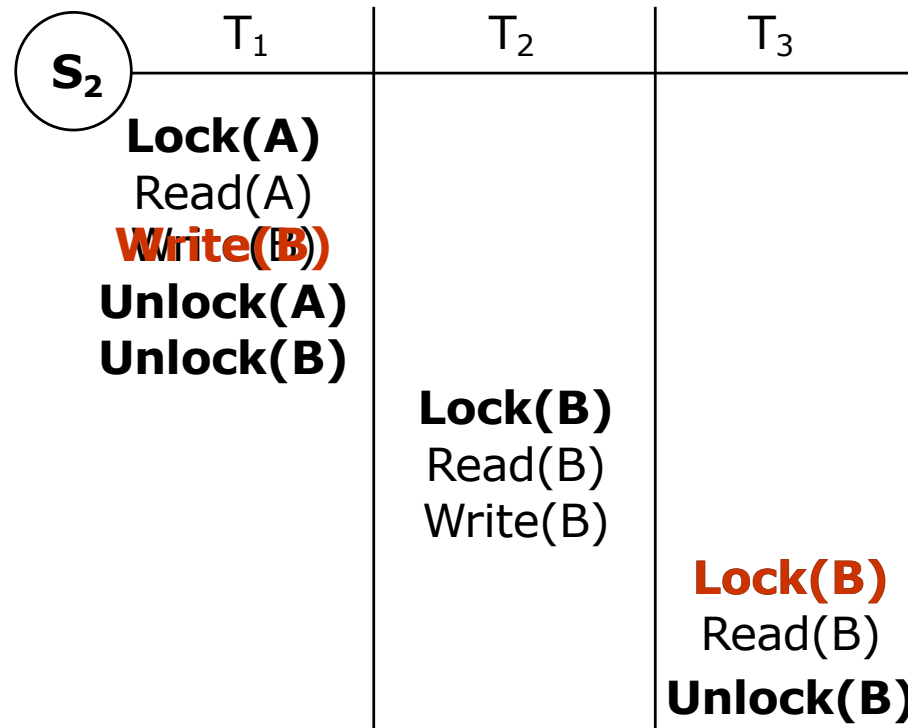
Exercise

- What are consistent transactions?
- Is the schedule legal?

	T ₁	T ₂	T ₃
S₁	Lock(A) Lock(B) Read(A) Write(B) Unlock(A) Unlock(B)	Lock(B) Read(B) Write(B) Unlock(B)	Lock(B) Read(B) Unlock(B)

Exercise

- What are consistent transactions?
- Is the schedule legal?



Exercise

- What are consistent transactions?
- Is the schedule legal?

	T ₁	T ₂	T ₃
S ₃	Lock(A) Read(A) Unlock(A) Lock(B) Write(B) Unlock(B)	Lock(B) Read(B) Write(B) Unlock(B)	Lock(B) Read(B) Unlock(B)

Discussion

- Suppose S is legal, is S serializable?

S	T ₁	T ₂	A	B
			25	25
	Lock(A); Read(A,t) t:=t+100 Write(A,t); Unlock(A)		125	
		Lock(A); Read(A,s) s:=s*2 Write(A,s); Unlock(A)	250	
		Lock(B); Read(B,s) s:=s*2 Write(B,s); Unlock(B)		50
	Lock(B); Read(B,t) t:=t+100 Write(B,t); Unlock(B)			150

Not serializable

Discussion

■ A simple change

- Lock B before releasing the lock on A

	T ₁	T ₂	A	B
S			25	25
Lock(A); Read(A,t)				
t:=t+100				
Write(A,t); Lock(B)			125	
Unlock(A)				
		Illegal schedule		
		Lock(A); Read(A,s)		
		s:=s*2		
		Write(A,s); Lock(B)		
		Unlock(A); Read(B,s)	250	
		s:=s*2		125
		Write(B,s); Unlock(B)		50
		Lock(B); Unlock(A);		
Read(B,t)		s:=s*2; Read(B,s)		
t:=t+100				
Write(B,t); Unlock(B)		Write(B,s); Unlock(B)		250

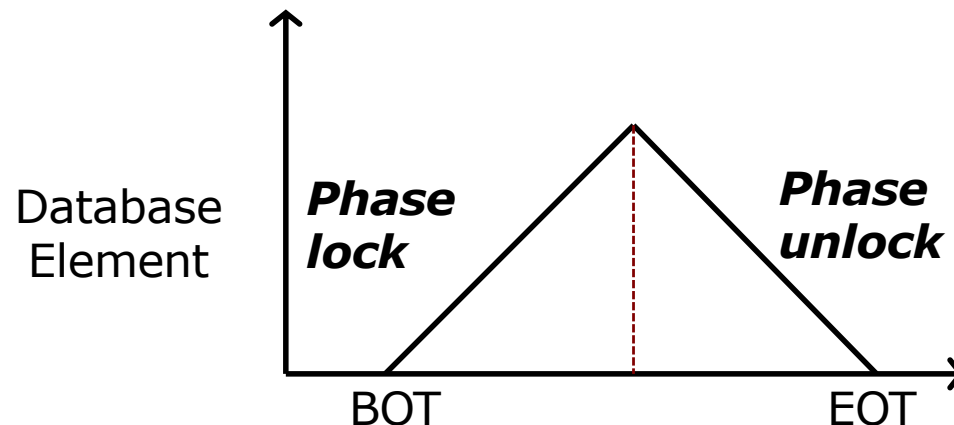
Two-phase locking

- A condition on the order of actions in a transaction
 - (3) *Two-phase locked transactions*

T_i : l_i(X) u_i(X) ...

←→

no unlockno lock****



Example

T ₁
Lock(A)
Read(A)
Lock(B)
Read(B)
B:=B+A
Write(B)
Unlock(A)
Unlock(B)

T ₂
Lock(B)
Read(B)
Lock(A)
Read(A)
Unlock(B)
A:=A+B
Write(A)
Unlock(A)

T ₃
Lock(B)
Read(B)
B=B-50
Write(B)
Unlock(B)
Lock(A)
Read(A)
A=A+50
Write(A)
Unlock(A)

T ₄
Lock(A)
Read(A)
Unlock(A)
Lock(B)
Read(B)
Unlock(B)
Print(A+B)

2PL transactions

Non-2PL transactions

Theorem

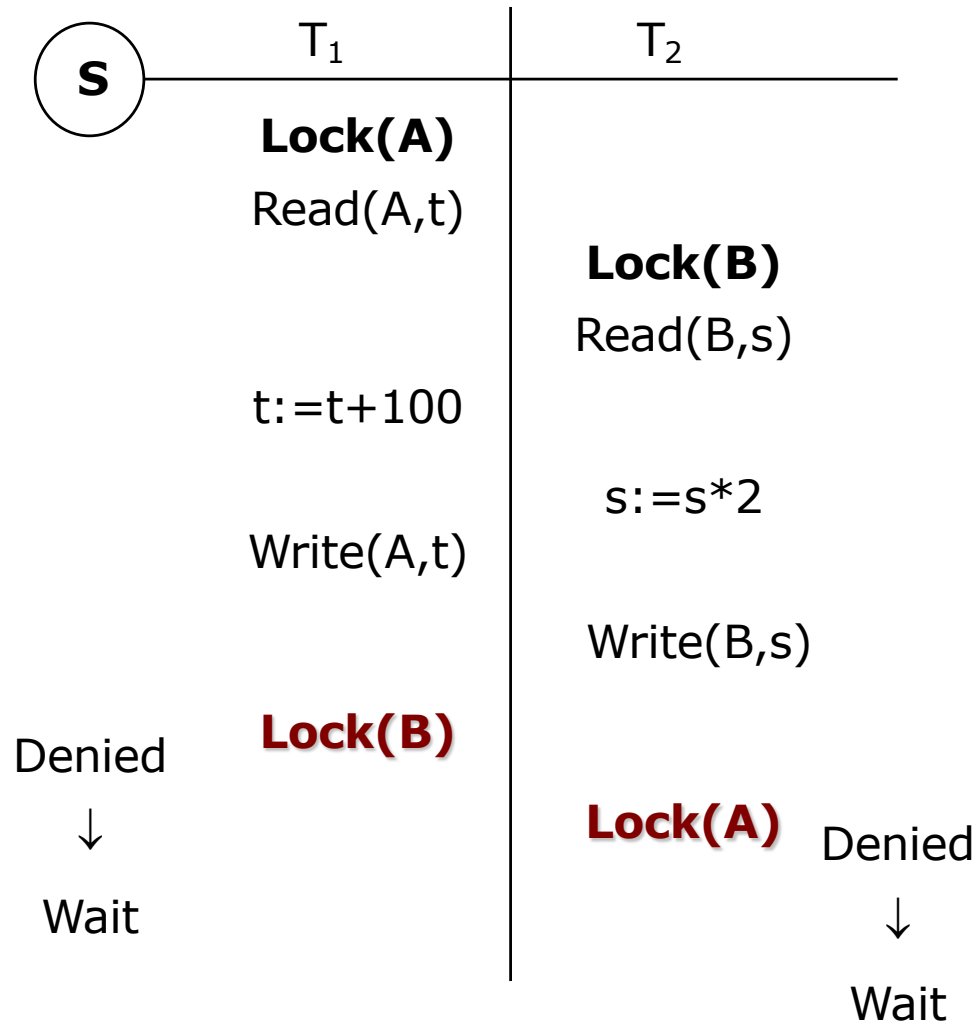
- A schedule S satisfies
 - (1) *Consistency of transactions*
 - (2) *Legality of schedules*
 - (3) *Two-phase locked transactions*
- S will be converted to a conflict-equivalent serial schedule

- Note
 - Non 2PL transactions allow not-conflict-serializable schedule

Proof

- Suppose $P(S)$ is cyclic
 - $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_n \rightarrow T_1$
- T_1 executes locks on elements which are unlocked by T_n
- T_1 has a sequence of actions as follows
 - Lock ... Unlock ... Lock
- This is illegal
 - Because all transactions in S are two-phase locked transactions
- $P(S)$ is acyclic

Note



Deadlock problem
is not solved by
two-phase locking

Discussion

■ Problem

- A transaction T must lock on element X , even if it only wants to read X and not write it
- But, we cannot avoid taking the lock
 - If not, unserializable behavior will happens
- Why many transactions could not read X at the same time, as long as none is allowed to write X ?

T_i	T_j
Lock(X) Read(X) Unlock(X)	Lock(X) Read(X) Unlock(X)

Lock for writing is “stronger” than lock for reading

Lock mode

- Locking scheduler uses 2 different kinds of locks
 - Shared lock
 - Read lock
 - $rl(X)$ or $R\text{Lock}(X)$
 - Exclusive lock
 - Write lock
 - $wl(X)$ or $W\text{Lock}(X)$
 - Release
 - Unlock
 - $u(X)$ or $U\text{lock}(X)$

Lock mode

- For any database element X
 - Either one exclusive lock on X
 - Or no exclusive locks but any number of shared locks
- If write(X)
 - Need to have WLock(X)
- If read(X)
 - May have either RLock(X) or WLock(X)

Lock mode

- Three kinds of requirements

- (1) Consistency of transactions

$T_i :$... **rl(X)/wl(X)** ... **r(X)** ... **u(X)** ...

$T_i :$... **wl(X)** ... **w(X)** ... **u(X)** ...

Lock mode

- Three kinds of requirements
 - (2) Legality of schedules

$$\text{S : } \quad \dots rl_i(X) \xrightarrow{\hspace{8em}} u_i(X) \dots$$

no $wl_j(X)$

S : ... $wl_i(A)$ $u_i(A)$...
 ↔
 no $wl_j(A)$
 no $rl_j(A)$

Lock mode

- Three kinds of requirements
 - (3) Two phase locking of transactions

T : ... rl(X)/wl(X) u(X) ...



no unlock no lock

Discussion

- Allow one transaction to request and hold both shared and exclusive lock on the same element

- If transactions know in advance their needs for locks

- Only exclusive lock

$T_i : \quad \dots wl(A) \dots r(A) \dots w(A) \dots u(A) \dots$

- If lock needs are unpredictable

- Both shared and exclusive locks at different times

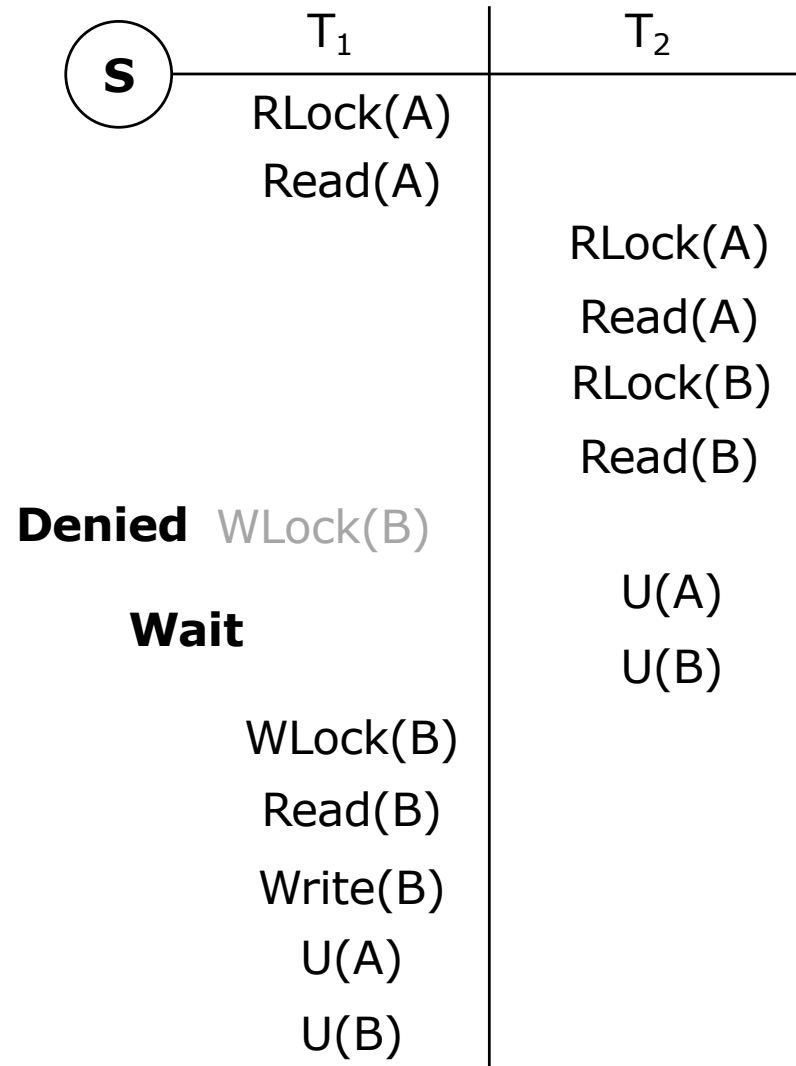
$T_i : \quad \dots rl(A) \dots r(A) \dots wl(A) \dots w(A) \dots u(A) \dots$

T_1

Read(A)

Write(A)?

Example



- S is conflict-serializable
- The order is (T₂, T₁)
 - Even though T₁ started first
 - But T₂ unlocks before T₁

Theorem

- In systems with shared and exclusive locks,
 - S must satisfy (1) & (2) & (3)
 - S is conflict-serializable

- Note
 - Non 2PL transactions allow not-conflict-serializable schedule

Compatibility matrix

- Using several lock modes
 - Scheduler needs a policy
 - When it can grant a lock request, given other locks that may already be held on the same database element

		Lock requested	
		Share	eXclusive
Lock held in mode	Share	yes	no
	eXclusive	no	no

Upgrading locks

- Examine a case
 - Transaction T take a ***shared lock*** on X
 - T wants to write a new value of X
 - When T is ready to write, ***upgrade the lock to exclusive***
 - Request an exclusive lock on X in addition to its already held shared lock on X
 - Nothing prevents a transaction from issuing requests for locks on the same DB element in different modes

Upgrading locks

- Examine a case

T_1

Read(A)
Write(A)?

- Solution

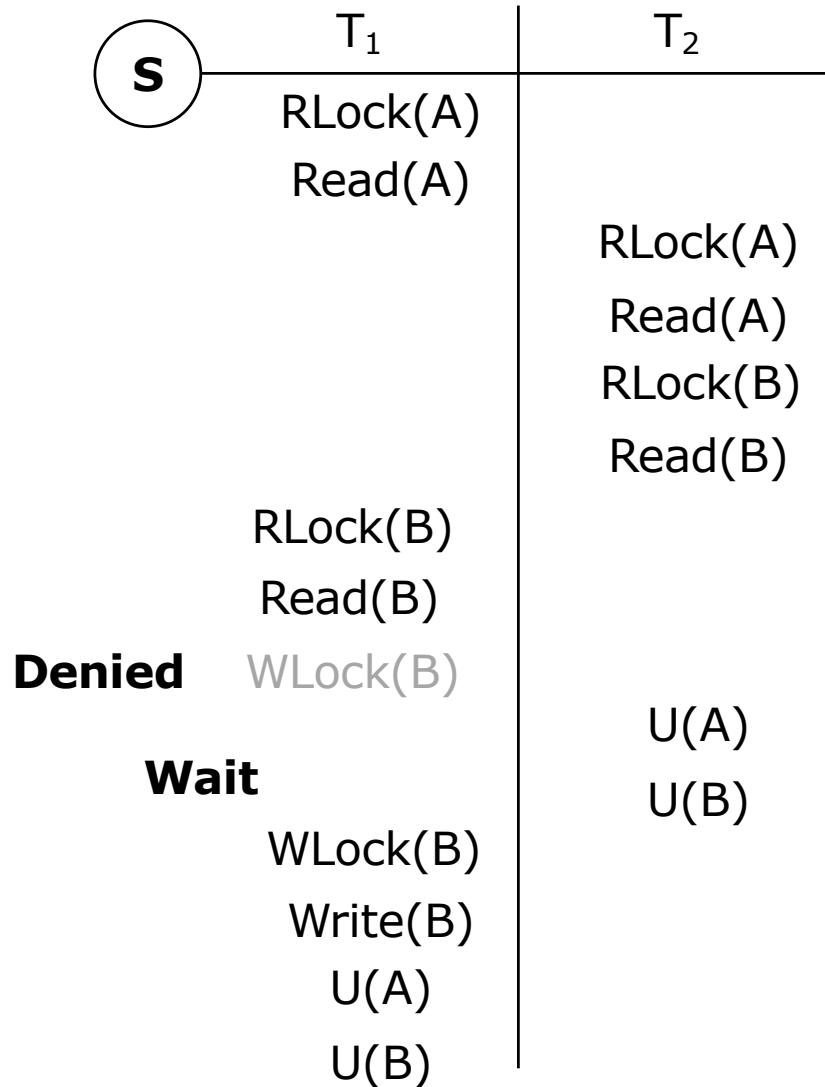
- (1) Request exclusive lock

$T_i : \dots \text{wl(A)} \dots \text{r(A)} \dots \text{w(A)} \dots \text{u(A)} \dots$

- (2) Upgrade exclusive lock

$T_i : \dots \text{rl(A)} \dots \text{r(A)} \text{u(A)} \text{wl(A)} \dots \text{w(A)} \dots \text{u(A)}$

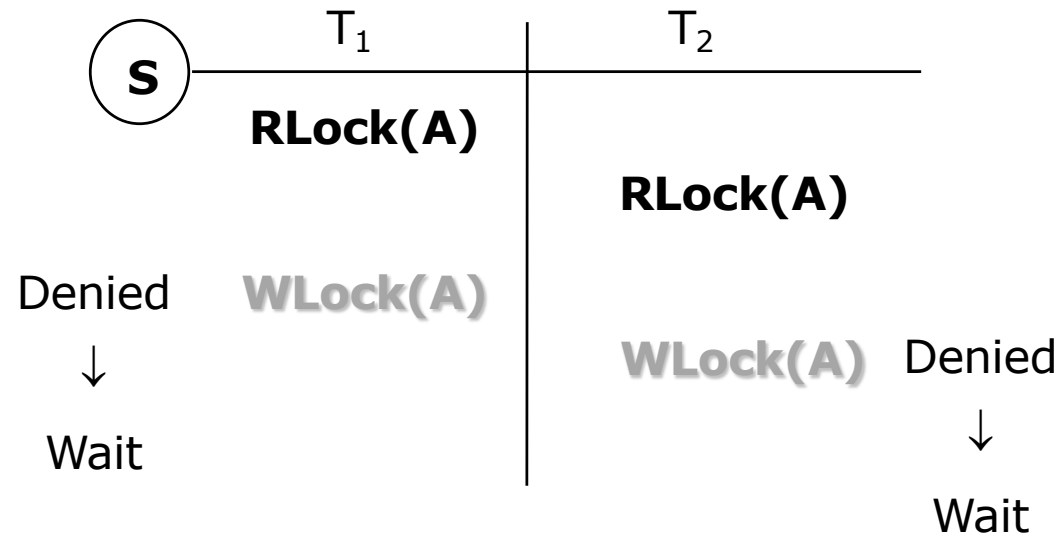
Upgrading locks



- T₂ gets a shared lock on B before T₁ does
- T₁ is also able to lock B in shared mode
- T₁ tried to upgrade its lock on B to exclusive

Upgrading locks

- Note



Upgrading by two transactions can cause a deadlock

Update locks

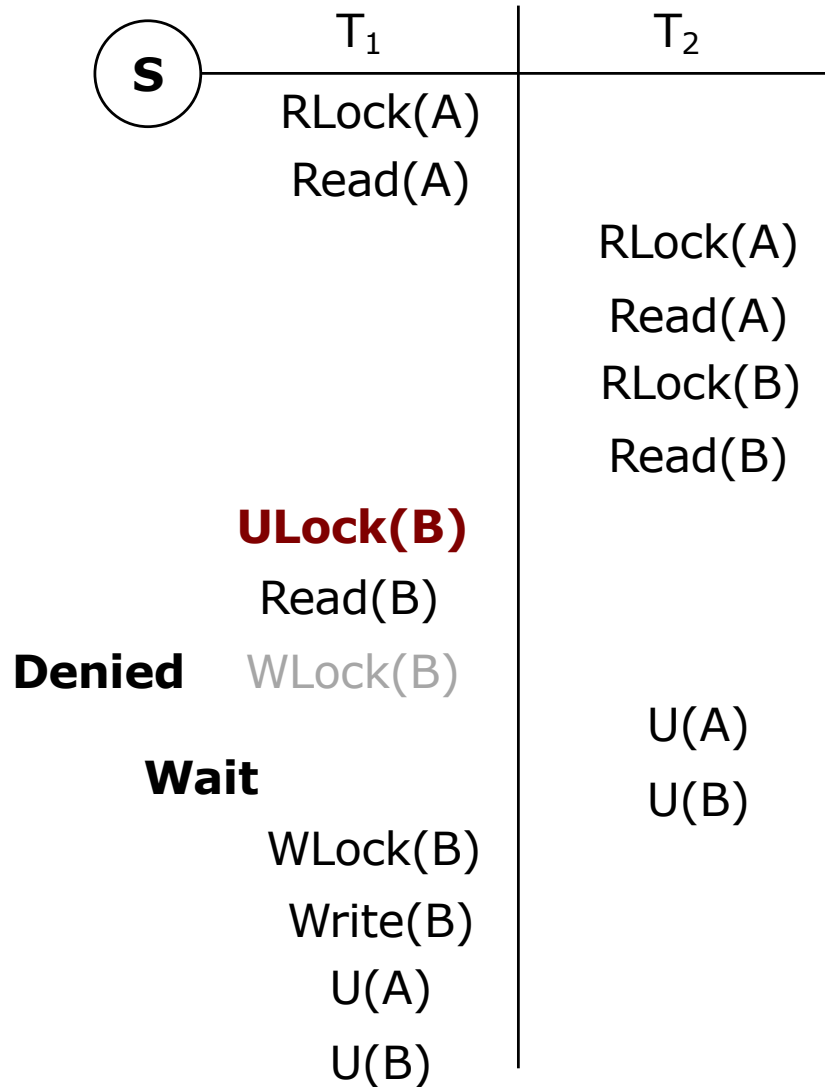
- Avoiding the deadlock
 - A third lock mode, ***update lock***
 - ul(X) or ULock(X)
 - Give the transaction T only the privilege to **read X, but not to write X**
 - Only update lock **can be upgraded to a write lock**, a read lock cannot be upgraded
 - Grant update lock on X when X was already locked by shared locks
 - Then, we prevent additional locks of any kind – shared, update, exclusive – on X

Update locks

- Compatibility matrix

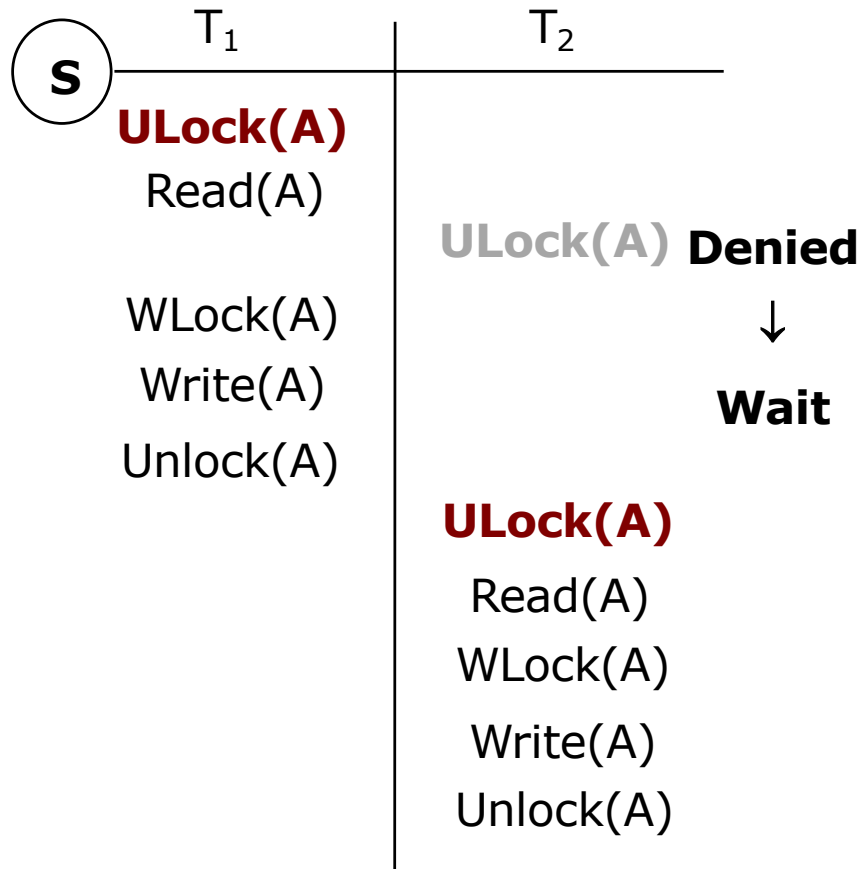
		Lock requested		
		Share	eXclusive	Update
Lock held in mode	Share	yes	no	yes
	eXclusive	no	no	no
	Update	no	no	no

Example



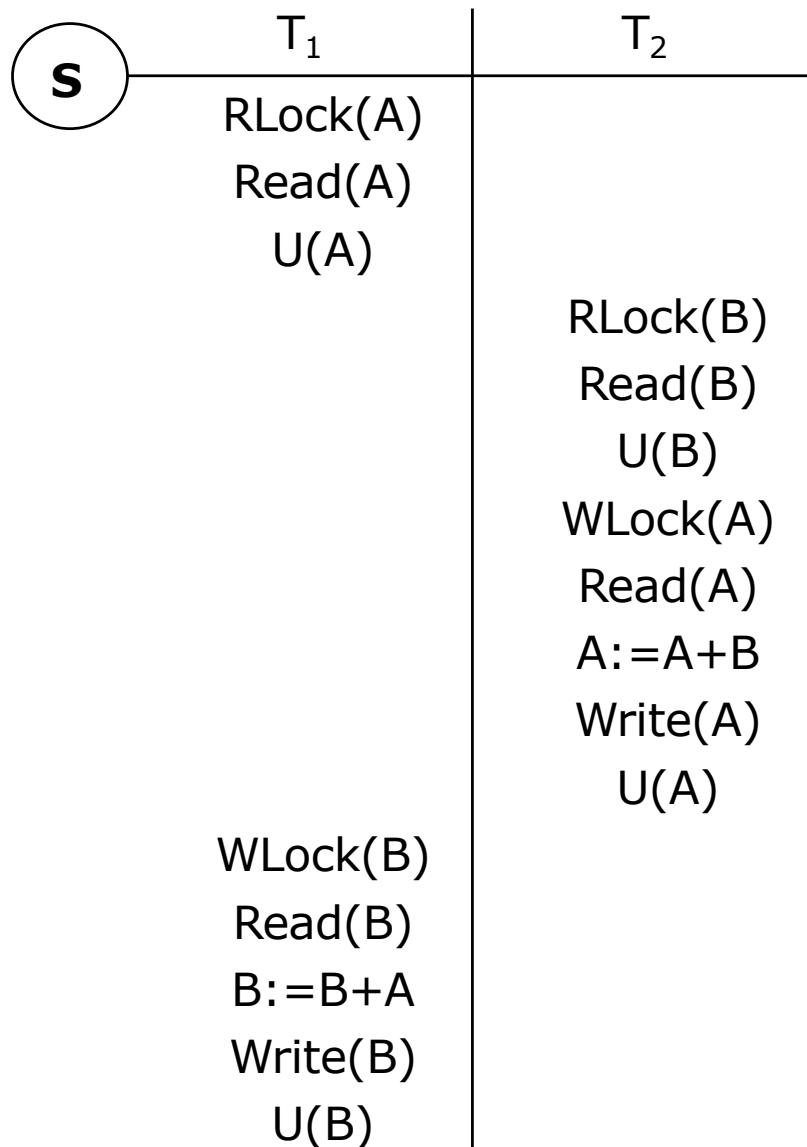
- T₁ would take an update lock on B, rather than a shared lock

Example



- Update locks fix the deadlock problem

Exercise



- Which transactions are not 2PL?
- Is S serializable?

Exercise

S	T ₁	T ₂	T ₃	T ₄
		RL(A)		
		WL(B)	RL(A)	
		U(A)		
		U(B)	WL(A)	
RL(B)			U(A)	
RL(A)				RL(B)
WL(C)				U(B)
U(A)				
U(B)				WL(B)
U(C)				U(B)

- Which transactions are not 2PL?
- Is S serializable?

Next lecture

- Recovery
- Read chapter 17 [1]

